V.Supriya

192472194

BE-CSE(AI)

CO2-AT2- Scenario Based Questions

Q3 – Spatial Filtering (Image Smoothing using Median Filter)

Description

Magnetic Resonance Imaging (MRI) is one of the most widely used medical imaging techniques for detecting brain abnormalities such as tumors. During the image acquisition process, MRI scans may be affected by **salt-and-pepper noise** caused by sensor interference, transmission errors, or environmental disturbances. This type of noise appears as random black and white pixels scattered across the image, making it difficult for doctors to clearly observe important brain structures.

To improve the quality of the MRI image, a **Median Filter** is applied. Median filtering is a non-linear image smoothing technique that replaces each pixel value with the median value of its neighboring pixels. Unlike averaging filters, the median filter effectively removes salt-and-pepper noise while preserving the edges and fine details of the image. This makes it especially suitable for medical images where maintaining structural information is critical for accurate diagnosis.

In this task, multiple Brain MRI images are processed using the Median Filter in OpenCV. The filtered images become smoother and clearer, allowing important anatomical features to remain visible while significantly reducing unwanted noise.

Dataset Link

Dataset Name: Brain MRI Images for Brain Tumor Detection

Source: Kaggle

Link: https://www.kaggle.com/datasets/navoneel/brain-mri-images-for-brain-tumor-detection?utm_source=chatgpt.com
https://www.kaggle.com/datasets/navoneel/brain-mri-images-for-brain-tumor-detection

Task

The objective of this task is to remove salt-and-pepper noise from Brain MRI images using the **Median Filtering** technique. The uploaded MRI images are processed one by one, and the filtered images are generated while preserving important brain edges and structures.

Input

- Brain MRI Images (Multiple Images)
- Image Format: JPG / PNG

- Dataset: Brain MRI Images for Brain Tumor Detection
- Number of Images Used: **3 MRI Images**

Example Input:

- MRI Image 1
- MRI Image 2
- MRI Image 3

Processing Steps

1. Upload the Brain MRI dataset ZIP file into Google Colab.
2. Automatically extract all the images from the ZIP file.
3. Read each MRI image using OpenCV.
4. Apply the Median Filter with a kernel size of **5 × 5** using cv2.medianBlur().
5. Display the Original MRI image and the Filtered MRI image side by side.
6. Save the filtered images into a separate output folder.

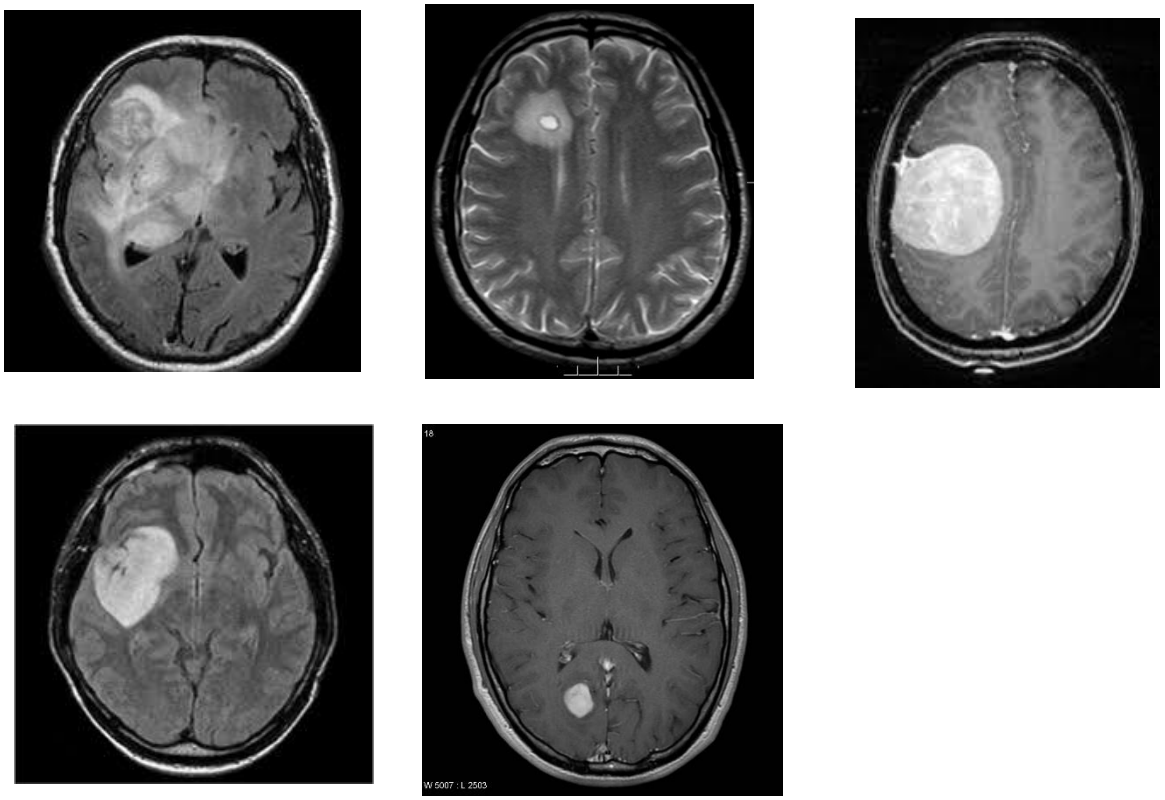
Tools Used

- Google Colab
- Python
- OpenCV (cv2)
- NumPy
- Matplotlib

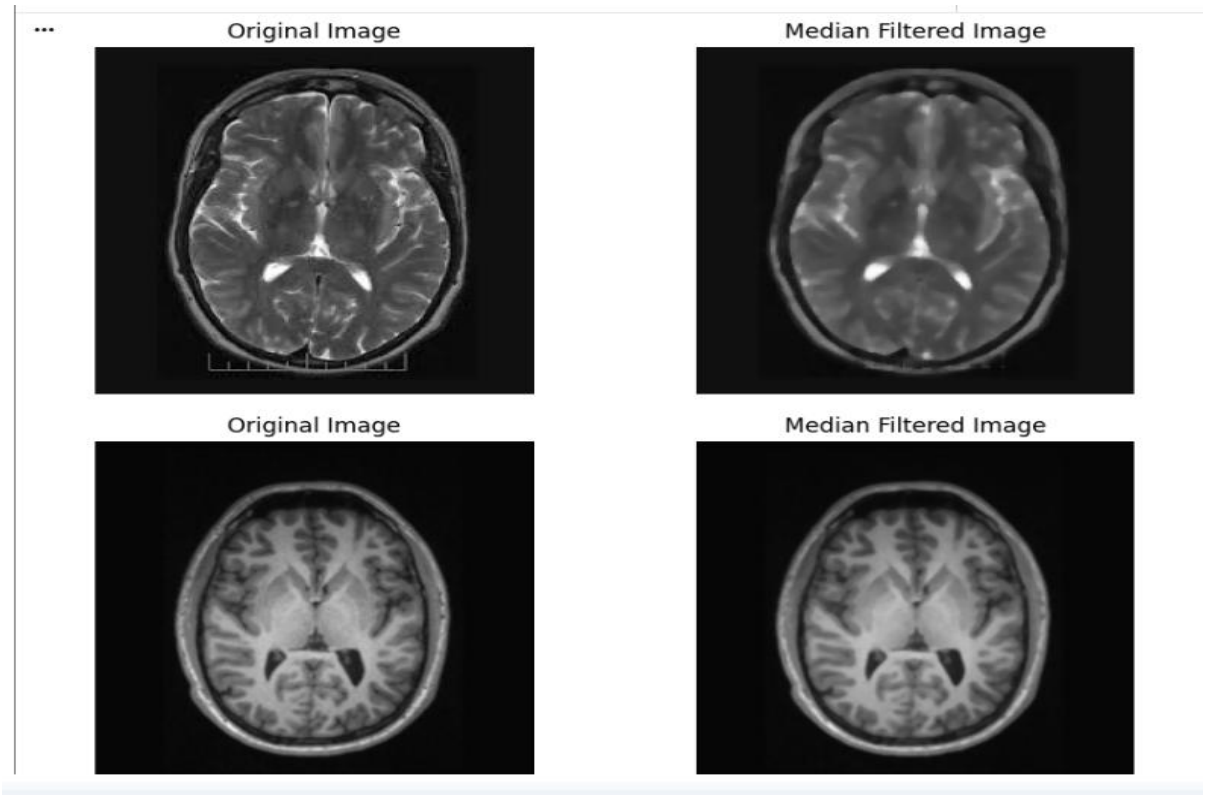
OpenCV Functions Used

- cv2.imread() – Reads the MRI image.
- cv2.medianBlur() – Removes salt-and-pepper noise using median filtering.
- cv2.cvtColor() – Converts the image from BGR to RGB for display.
- cv2.imwrite() – Saves the filtered image.

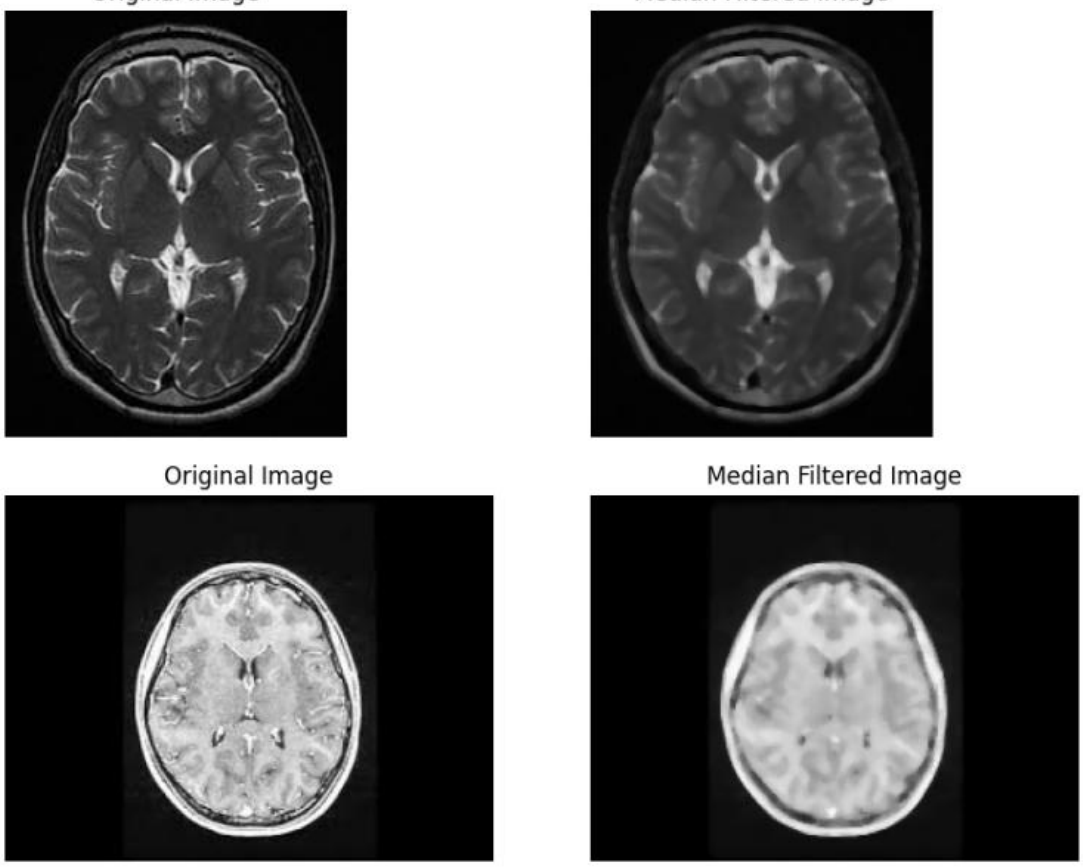
INPUT IMAGES



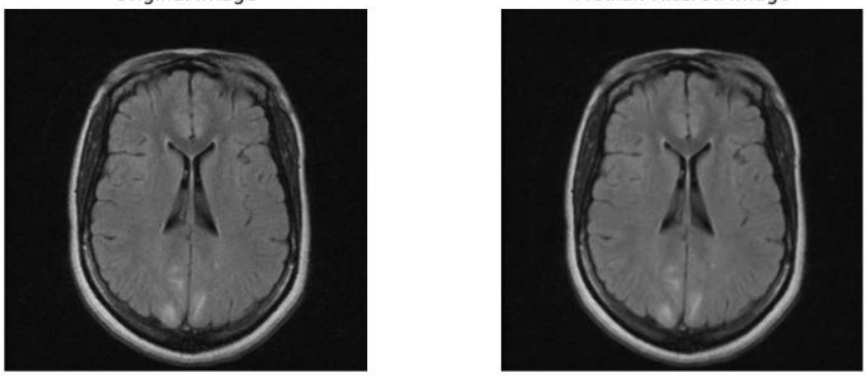
OUTPUT IMAGES



...


Original Image
Median Filtered Image
Original Image
Median Filtered Image

...


Original Image
Median Filtered Image

Q4 – Spatial Filtering (Image Sharpening using Laplacian Operator)

Description

In infrastructure inspection, drones are commonly used to capture images of concrete bridges, buildings, and roads to identify structural defects such as cracks. However, due to slight camera motion, poor lighting, or image blur, the cracks may appear faint and unclear. This makes it difficult for engineers and automated inspection systems to accurately detect and measure the damaged areas.

To enhance the visibility of these fine cracks, Image Sharpening is performed using the Laplacian Operator. The Laplacian is a second-order derivative operator that highlights regions with rapid intensity changes, such as edges and boundaries. It detects high-frequency details in the image, making cracks more prominent. After computing the Laplacian image, the detected edges are subtracted from the original grayscale image to produce a sharpened image with clearer crack boundaries.

In this task, multiple concrete crack images are processed using the Laplacian sharpening technique in OpenCV. The resulting sharpened images provide better edge definition, making it easier for structural inspection and crack analysis.

Dataset Link

Dataset Name: Concrete Crack Images for Classification

Source: Kaggle

Link: https://www.kaggle.com/datasets/arnavr10880/concrete-crack-images-for-classification?utm_source=chatgpt.com
https://www.kaggle.com/datasets/arnavr10880/concrete-crack-images-for-classification

Task

The objective of this task is to enhance the visibility of fine cracks in concrete images using the Laplacian Image Sharpening technique. The uploaded crack images are processed individually to produce sharper images with more distinct edges.

Input

- Multiple Concrete Crack Images
- Image Format: JPG / PNG
- Dataset: Concrete Crack Images for Classification
- Number of Images Used: 3 Crack Images

Example Input:

- Crack Image 1
- Crack Image 2

- Crack Image 3

Processing Steps

1. Upload the Concrete Crack dataset ZIP file into Google Colab.
2. Automatically extract all the images from the ZIP file.
3. Read each crack image in grayscale using OpenCV.
4. Apply the Laplacian Operator using:

cv2.Laplacian(image, cv2.CV_16S, ksize=3)

5. Convert the Laplacian output to an 8-bit image using cv2.convertScaleAbs().
6. Subtract the Laplacian image from the original image using cv2.subtract() to obtain the sharpened image.
7. Display the original and sharpened images side by side.
8. Save the sharpened images in a separate output folder.

Tools Used

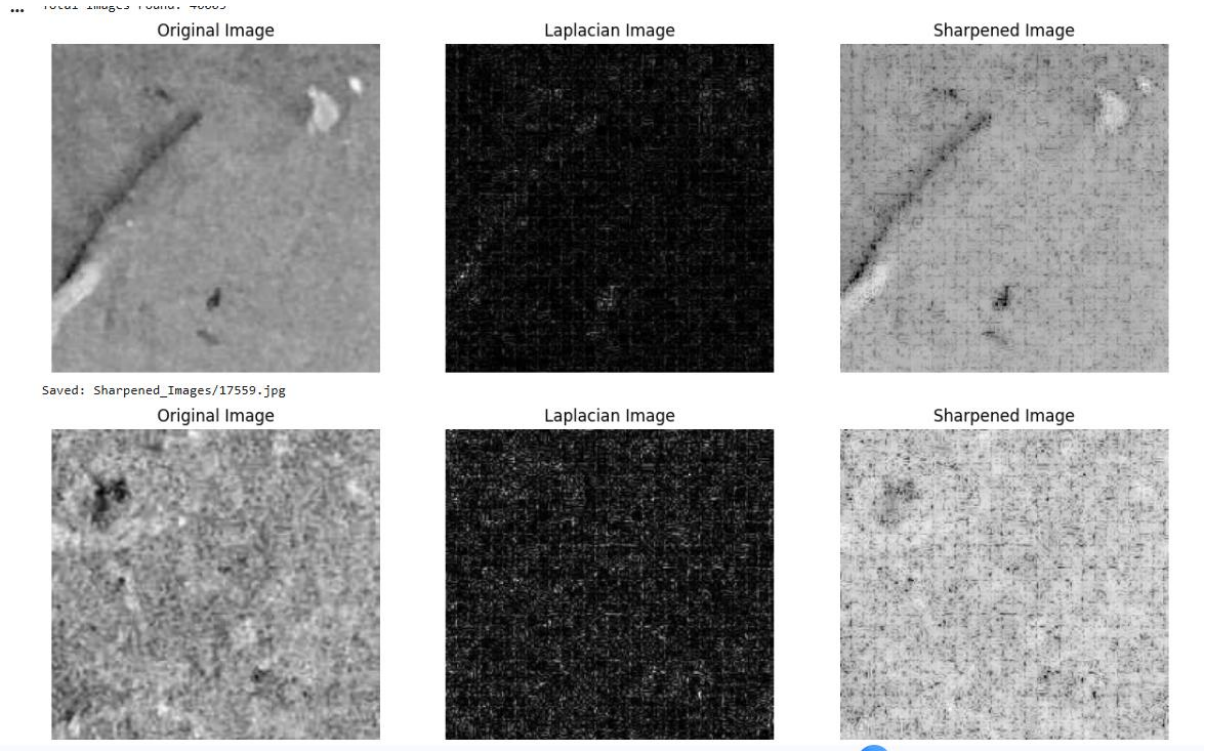
- Google Colab
- Python
- OpenCV (cv2)
- NumPy
- Matplotlib

OpenCV Functions Used

- cv2.imread() – Reads the crack image.
- cv2.Laplacian() – Detects edges and fine details.
- cv2.convertScaleAbs() – Converts the Laplacian result into an 8-bit image.
- cv2.subtract() – Sharpens the image by subtracting the Laplacian image.
- cv2.imwrite() – Saves the sharpened image.

INPUT IMAGES

OUTPUT IMAGES



Saved: Sharpened_Images/17559.jpg